

LINQ GroupBy Method with Multiple Keys in C#

In this article, I will discuss the **LINQ GroupBy Method with Multiple Keys in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ GroupBy Method in C#](#) with Examples. As part of this article, we will discuss the following pointers.

1. [Why Do We Need to Group the Data Based on Multiple Keys?](#)
2. [How Do We Use the LINQ GroupBy Method with Multiple Keys in C#?](#)
3. [Multiple Real-Time Examples using both Method and Query Syntax.](#)
4. [When Should We Use LINQ GroupBy Method with Multiple Keys?](#)

Why do we need to group the data based on Multiple Keys?

It is a common requirement in real-time applications to group the data based on multiple keys. In LINQ (Language-Integrated Query), you can use the GroupBy method to group elements in a collection based on one or more keys. This approach is useful when organizing data based on a combination of properties.

The GroupBy Method in LINQ (Language Integrated Query) allows us to group sequence elements based on a specified key. When you want to group by multiple keys, you typically use an anonymous type or a tuple (in newer versions of C#, i.e., from C# 7.1) because these types provide a convenient way to combine multiple fields into a single object that can be used as a key.

LINQ GroupBy Method in C# with Examples using Multiple Keys

Let us understand how to use the LINQ GroupBy Method in C# with Multiple Keys. We will use the following Student class to understand the GroupBy Method with Multiple Keys. So, create a class file named **Student.cs** and copy and paste the following code. This class has five properties: ID, Name, Gender, Branch, and Age. This class also has one method called GetStudents(), which returns a list of all students and will be our data source.

```
using System.Collections.Generic;
namespace GroupByDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public string Branch { get; set; }
        public int Age { get; set; }
        public static List<Student> GetStudents()
        {
            return new List<Student>()
            {
                new Student { ID = 1001, Name = "Preety", Gender = "Female", Branch = "CSE", Age = 20 },
                new Student { ID = 1002, Name = "Snurag", Gender = "Male", Branch = "ETC", Age = 21 },
            }
        }
    }
}
```

```

        new Student { ID = 1003, Name = "Pranaya", Gender = "Male", Branch = "CSE", Age = 21 },
        new Student { ID = 1004, Name = "Anurag", Gender = "Male", Branch = "CSE", Age = 20 },
        new Student { ID = 1005, Name = "Hina", Gender = "Female", Branch = "ETC", Age = 20 },
        new Student { ID = 1006, Name = "Priyanka", Gender = "Female", Branch = "CSE", Age = 21 },
        new Student { ID = 1007, Name = "santosh", Gender = "Male", Branch = "CSE", Age = 22 },
        new Student { ID = 1008, Name = "Tina", Gender = "Female", Branch = "CSE", Age = 20 },
        new Student { ID = 1009, Name = "Celina", Gender = "Female", Branch = "ETC", Age = 22 },
        new Student { ID = 1010, Name = "Sambit", Gender = "Male", Branch = "CSE", Age = 21 }
    };
}
}
}

```

Example to Understand Grouping Students Based on Multiple Keys in C#

We want to group the students based on Branch and gender. First, we need to group the students by Branch, and then we need to group the students by Gender. Also, we need to sort the students in each group by Name in Ascending order. For a better understanding, please look at the following example, which exactly does the same thing. In the below example, using an anonymous object, we are specifying two keys, i.e., **new { x.Branch, x.Gender }**. The following example code is self-explained, so please go through the comment lines.

```

using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var GroupByMultipleKeysMS = Student.GetStudents()
                .GroupBy(x => new { x.Branch, x.Gender })
                .Select(g => new
                {
                    Branch = g.Key.Branch,
                    Gender = g.Key.Gender,
                    Students = g.OrderBy(x => x.Name)
                });
            //Using Query Syntax
            var GroupByMultipleKeysQS = (from std in Student.GetStudents()

```

Object

//Grouping Multiple Keys using an Anc

```
group std by new
{
    std.Branch,
    std.Gender
} into stdGroup
select new
{
    Branch = stdGroup.Key.Branch,
    Gender = stdGroup.Key.Gender,
    //Sort the Students of Each group
```

Ascending Order

```
Students = stdGroup.OrderBy(x =>
    ));
//It will iterate through each group
foreach (var group in GroupByMultipleKeysQS)
{
    Console.WriteLine($"Branch : {group.Branch} Gender: {group.Ger
Students = {group.Students.Count()}");
    //It will iterate through each item of a group
    foreach (var student in group.Students)
    {
        Console.WriteLine($" ID: {student.ID}, Name: {student.Nar
{student.Age} ");
    }
    Console.WriteLine();
}
Console.Read();
}
}
}
```

When you run the above code, you will get the following output.

```
Barnch : CSE Gender: Female No of Students = 3
ID: 1001, Name: Preety, Age: 20
ID: 1006, Name: Priyanka, Age: 21
ID: 1008, Name: Tina, Age: 20
```

```
Barnch : ETC Gender: Male No of Students = 2
ID: 1010, Name: Sambit, Age: 21
ID: 1002, Name: Snurag, Age: 21
```

```
Barnch : CSE Gender: Male No of Students = 3
ID: 1004, Name: Anurag, Age: 20
ID: 1003, Name: Pranaya, Age: 21
ID: 1007, Name: santosh, Age: 22
```

```
Barnch : ETC Gender: Female No of Students = 2
ID: 1009, Name: Celina, Age: 22
ID: 1005, Name: Hina, Age: 20
```

Using Tuples (C# 7.1 and later)

Let us rewrite the previous example to use the Tuples and LINQ GroupBy method for grouping by multiple keys. Tuples are a convenient way to group multiple values without defining a separate class or structure. Instead of creating an anonymous type with new { **x.Branch**, **x.Gender** }, the following example uses a Tuple (**student.Branch**, **student.Gender**) for grouping. Tuples are a more concise and readable way to handle multiple keys for grouping. Please modify the Program class as follows:

```
using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var GroupByMultipleKeysMS = Student.GetStudents()
                //Grouping Multiple Keys using using Tuples
                .GroupBy(student => (student.Branch, student.Gender))
                .Select(g => new
                {
                    Branch = g.Key.Branch,
                    Gender = g.Key.Gender,
                    Students = g.OrderBy(x => x.Name)
                }); ;
            //Using Query Syntax
            var GroupByMultipleKeysQS = (from std in Student.GetStudents()
                //Grouping Multiple Keys using Tuples
                group std by (std.Branch, std.Gender)
                into stdGroup
                select new
```

```

        {
            Branch = stdGroup.Key.Branch,
            Gender = stdGroup.Key.Gender,
            //Sort the Students of Each group
            Students = stdGroup.OrderBy(x =>
                x.Age);
        }
        //It will iterate through each group
        foreach (var group in GroupByMultipleKeysQS)
        {
            Console.WriteLine($"Branch : {group.Branch} Gender: {group.Gender}");
            Students = {group.Students.Count()};
            //It will iterate through each item of a group
            foreach (var student in group.Students)
            {
                Console.WriteLine($" ID: {student.ID}, Name: {student.Name}, Age: {student.Age} ");
            }
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}

```

Now, run the application; you should get the same output as the previous example.

Notes:

- Anonymous Types are useful for quick and simple groupings, especially when the grouping logic won't be reused elsewhere.
- Tuples offer a bit more flexibility and are useful when you want to pass grouping keys around or use them in other parts of your code since they are actual types rather than just compiler-generated ones.

Grouping Students Based on the Branch and Gender along with OrderBy Method

Now, we want to group the students based on Branch and gender. First, we need to group the students by Branch in Descending Order, and then we need to group the students by Gender in Ascending order on each branch group. Finally, the students in each group need to be sorted by their names in Ascending Order. For a better understanding, please look at the following example, which does exactly the same thing. The following example code is self-explained, so please go through the comment lines.

```

using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

//Using Method Syntax
var GroupByMultipleKeysMS = Student.GetStudents()
//Group the Students first by Branch and then by Gender
.GroupBy(x => new { x.Branch, x.Gender })
//Sort Each Group in Descending Order
.OrderByDescending(g => g.Key.Branch)
//Then Sort Each Branch Group in Ascending Order
.OrderBy(g => g.Key.Gender)
//Project the Result to an Anonymous Type
.Select(g => new
{
    Branch = g.Key.Branch,
    Gender = g.Key.Gender,
    //Sort the Students of Each group
    Students = g.OrderBy(x => x.Name)
});

//Using Query Syntax
var GroupByMultipleKeysQS = from student in Student.GetStudents()
//Group the Students by Branch and then by Gender
//Result into a variable
group student by new
{
    student.Branch,
    student.Gender
} into stdGroup
//Then Sort the group by Branch Descending and Gender Ascending
orderby stdGroup.Key.Branch descending,
stdGroup.Key.Gender ascending
//Project the Result to an Anonymous Type
select new
{
    Branch = stdGroup.Key.Branch,
    Gender = stdGroup.Key.Gender,
    //Sort the Students of Each group
    Students = stdGroup.OrderBy(x => x.Name)
};

//It will iterate through each group
foreach (var group in GroupByMultipleKeysQS)
{
    Console.WriteLine($"Branch : {group.Branch} Gender: {group.Gender}");
    Students = {group.Students.Count()};
    //It will iterate through each item of a group
    foreach (var student in group.Students)
    {

```

```

        Console.WriteLine($" ID: {student.ID}, Name: {student.Nar
{student.Age} ");
    }
    Console.WriteLine();
}
Console.Read();
}
}
}
}

```

You will get the following output when you run the above application code.

```

Barnch : ETC Gender: Female No of Students = 2
ID: 1009, Name: Celina, Age: 22
ID: 1005, Name: Hina, Age: 20

Barnch : ETC Gender: Male No of Students = 2
ID: 1010, Name: Sambit, Age: 21
ID: 1002, Name: Snurag, Age: 21

Barnch : CSE Gender: Female No of Students = 3
ID: 1001, Name: Preety, Age: 20
ID: 1006, Name: Priyanka, Age: 21
ID: 1008, Name: Tina, Age: 20

Barnch : CSE Gender: Male No of Students = 3
ID: 1004, Name: Anurag, Age: 20
ID: 1003, Name: Pranaya, Age: 21
ID: 1007, Name: santosh, Age: 22

```

We have projected the result to an anonymous type in the above example. You can also create a new type with the required properties and then project the result to that new type. Let's understand this. First, create a class file named **StudentGroupByBranchGender.cs**, and copy and paste the following code.

```

using System.Collections.Generic;
namespace GroupByDemo
{
    public class StudentGroupByBranchGender
    {
        public string Branch { get; set; }
        public string Gender { get; set; }
        public List<Student> Students { get; set; }
    }
}

```

With the above changes in place, modify the Main method of the Program class as follows. Here, we project the result into the newly created **StudentGroupByBranchGender** type instead of the anonymous one.

```

using System;

```

```

using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var GroupByMultipleKeysMS = Student.GetStudents()
                //Group the Students first by Branch and then by Gender
                .GroupBy(x => new { x.Branch, x.Gender })
                //Sort Each Group in Descending Order
                .OrderByDescending(g => g.Key.Branch)
                //Then Sort Each Branch Group in Ascending Order
                .ThenBy(g => g.Key.Gender)
                //Project the Result to StudentGroupByBranchGender Type
                .Select(g => new StudentGroupByBranchGender
                {
                    Branch = g.Key.Branch,
                    Gender = g.Key.Gender,
                    //Sort the Students of Each group in Ascending Order
                    Students = g.OrderBy(x => x.Name).ToList();
                });

            //Using Query Syntax
            var GroupByMultipleKeysQS = from student in Student.GetStudents()
                //Group the Students by Branch and then by Gender
                //Result into a variable
                group student by new
                {
                    student.Branch,
                    student.Gender
                } into stdGroup
                //Then Sort the group by Branch Descending and Gender Ascending
                orderby stdGroup.Key.Branch descending, stdGroup.Key.Gender ascending
                //Project the Result to StudentGroupByBranchGender Type
                select new StudentGroupByBranchGender
                {
                    Branch = stdGroup.Key.Branch,
                    Gender = stdGroup.Key.Gender,
                    //Sort the Students of Each group in Ascending Order

```

```

Students = stdGroup.OrderBy(x =>
x.Name).ToList()
    };
    //It will iterate through each group
    foreach (StudentGroupByBranchGender group in GroupByMultipleKeysQs)
    {
        Console.WriteLine($"Barnch : {group.Branch} Gender: {group.Ger
Students = {group.Students.Count()}");
        //It will iterate through each item of a group
        foreach (var student in group.Students)
        {
            Console.WriteLine($" ID: {student.ID}, Name: {student.Nar
{student.Age} ");
        }
        Console.WriteLine();
    }
    Console.Read();
}
}
}

```

Now, run the application code, and you will get the following output as expected.

```

Barnch : ETC Gender: Female No of Students = 2
  ID: 1009, Name: Celina, Age: 22
  ID: 1005, Name: Hina, Age: 20

Barnch : ETC Gender: Male No of Students = 2
  ID: 1010, Name: Sambit, Age: 21
  ID: 1002, Name: Snurag, Age: 21

Barnch : CSE Gender: Female No of Students = 3
  ID: 1001, Name: Preety, Age: 20
  ID: 1006, Name: Priyanka, Age: 21
  ID: 1008, Name: Tina, Age: 20

Barnch : CSE Gender: Male No of Students = 3
  ID: 1004, Name: Anurag, Age: 20
  ID: 1003, Name: Pranaya, Age: 21
  ID: 1007, Name: santosh, Age: 22

```

When Should We Use the LINQ GroupBy Method with Multiple Keys?

Here are scenarios when you might use GroupBy with multiple keys:

- **Complex Data Structures:** When working with complex data with multiple dimensions or attributes, grouping by multiple keys can help organize the data more effectively. This is common in financial,

statistical, or business analysis applications where data might need to be grouped by time period and category or type.

- **Detailed Data Analysis:** For in-depth analysis that requires data to be examined from multiple perspectives, using multiple keys for grouping can provide a finer granularity of data segmentation. For instance, in customer data analysis, you might group data by region and customer segment to analyze purchasing patterns.
- **Data Aggregation Across Dimensions:** When you need to aggregate data across several dimensions, grouping by multiple keys allows for more sophisticated and detailed aggregations. For example, in sales data, you might want to calculate total sales by product category and store location.
- **Handling Hierarchical Data:** When dealing with hierarchical or nested data, grouping by multiple keys can help organize the data according to its hierarchy. For example, in an organizational chart, employees could be grouped by department and then by role.

Complex Data Structures Example using LINQ GroupBy Method with Multiple Keys

The GroupBy method is useful for organizing elements into groups based on some keys that we need to specify. When dealing with complex data structures, you might often find the need to group by multiple keys. This can be achieved using anonymous types or tuples as the key selector in the GroupBy method.

Here's an example that demonstrates how to use the GroupBy method with multiple keys on a complex data structure. Suppose we have a collection of Order objects, where each Order has a CustomerId, OrderId, OrderDate, and Total amount. We want to group these orders by both CustomerId and OrderYear (extracted from OrderDate).

First, create a class file named **Order.cs** and copy and paste the following code. We have also defined one method to return a list of orders.

```
using System;
using System.Collections.Generic;
namespace GroupByDemo
{
    public class Order
    {
        public int CustomerId { get; set; }
        public int OrderId { get; set; }
        public DateTime OrderDate { get; set; }
        public decimal Total { get; set; }

        public static List<Order> GetAllOrders()
        {
            // Sample list of orders
            List<Order> orders = new List<Order>
            {
                new Order { CustomerId = 1, OrderId = 100, OrderDate = new DateTime(2015, 1, 25), Total = 150.00m },
                new Order { CustomerId = 1, OrderId = 101, OrderDate = new DateTime(2015, 3, 12), Total = 250.00m },
                new Order { CustomerId = 2, OrderId = 102, OrderDate = new DateTime(2015, 2, 28), Total = 100.00m },
            }
        }
    }
}
```

```

        new Order { CustomerId = 2, OrderId = 103, OrderDate = new DateTime(2011, 1, 15), Total = 200.00m },
        // Add more orders as needed
    };
    return orders;
}
}
}

```

Now, let's group the orders by CustomerId and OrderYear. Modify the Program class as follows:

```

using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Grouping by CustomerId and OrderYear
            var groupedOrders = Order.GetAllOrders()
                .GroupBy(order => new
                {
                    order.CustomerId,
                    OrderYear = order.OrderDate.Year
                })
                .Select(group => new
                {
                    CustomerId = group.Key.CustomerId,
                    OrderYear = group.Key.OrderYear,
                    TotalOrders = group.Count(),
                    TotalAmount = group.Sum(order => order.Total)
                });
            // Displaying the grouped information
            foreach (var group in groupedOrders)
            {
                Console.WriteLine($"CustomerId: {group.CustomerId}, OrderYear: {group.OrderYear}, Orders: {group.TotalOrders}, TotalAmount: {group.TotalAmount}");
            }
            Console.Read();
        }
    }
}

```

In this example:

- We group the Order objects by a composite key consisting of CustomerId and OrderYear. This is done using an anonymous type in the GroupBy method.
- The Select method is then used to project each group into a new anonymous type that includes the CustomerId, OrderYear, the count of orders in each group (TotalOrders), and the sum of the Total amount for all orders in each group (TotalAmount).

- Finally, we iterate over each group and print out the grouping information, including the total number of orders and the total amount for each group.

When you run the above code, you will get the following output:

```
CustomerId: 1, OrderYear: 2023, Orders: 2, TotalAmount: 400.00
CustomerId: 2, OrderYear: 2023, Orders: 1, TotalAmount: 100.00
CustomerId: 2, OrderYear: 2024, Orders: 1, TotalAmount: 200.00
```

Detailed Data Analysis Example using LINQ GroupBy Method with Multiple Keys

Let's see an example of performing data analysis using the LINQ GroupBy method in C# with multiple keys. Grouping data is a common operation in data analysis, allowing you to categorize data into groups and perform operations like counting, summing, or averaging on each group. Suppose we have a collection of sales records for a company, and each record contains the following information:

- **SaleId:** A unique identifier for the sale.
- **ProductId:** The ID of the product sold.
- **Country:** The country where the sale was made.
- **Year:** The year the sale was made.
- **Amount:** The amount of money made from the sale.

Our goal is to analyze these sales records by grouping them based on Country and Year and then summarizing the total sales amount and the sales count in each group.

First, we define a class to represent the sales records. Create a class file named **SaleRecord.cs** and copy and paste the following code. We have also defined one method that will return a list of sales.

```
using System.Collections.Generic;
namespace GroupByDemo
{
    public class SaleRecord
    {
        public int SaleId { get; set; }
        public int ProductId { get; set; }
        public string Country { get; set; }
        public int Year { get; set; }
        public decimal Amount { get; set; }
        public static List<SaleRecord> GetAllSales()
        {
            var sales = new List<SaleRecord>
            {
                new SaleRecord { SaleId = 1, ProductId = 101, Country = "USA",
2023, Amount = 150M },
                new SaleRecord { SaleId = 2, ProductId = 102, Country = "USA",
2023, Amount = 250M },
            }
        }
    }
}
```

```

                new SaleRecord { SaleId = 3, ProductId = 103, Country = "Canada",
2023, Amount = 100M },
                new SaleRecord { SaleId = 4, ProductId = 104, Country = "USA",
2023, Amount = 150M },
                new SaleRecord { SaleId = 5, ProductId = 102, Country = "Canada",
2023, Amount = 170M },
                new SaleRecord { SaleId = 6, ProductId = 101, Country = "India",
2023, Amount = 550M },
                new SaleRecord { SaleId = 1, ProductId = 101, Country = "USA",
2024, Amount = 150M },
                new SaleRecord { SaleId = 2, ProductId = 102, Country = "USA",
2024, Amount = 250M },
                new SaleRecord { SaleId = 3, ProductId = 103, Country = "Canada",
2024, Amount = 100M },
                // Add more records as needed
            };

            return sales;
        }
    }
}

```

We use an anonymous type in the GroupBy method to group by multiple keys. So, modify the Program class as follows:

```

using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //To group by multiple keys, we use an anonymous type in the GroupBy method
            var groupedSales = SaleRecord.GetAllSales().GroupBy(s => new { s.Country,
s.Year });

            groupedSales.Select(g => new
            {
                Country = g.Key.Country,
                Year = g.Key.Year,
                TotalAmount = g.Sum(x => x.Amount),
                SaleCount = g.Count()
            });

            // Analyzing the Results
            //we can iterate over the grouped results to analyze the total sales and
            //and count of sales for each country and year
            foreach (var group in groupedSales)
            {
                Console.WriteLine($"Country: {group.Country}, Year: {group.Year},
Sales: {group.TotalAmount}, Sales Count: {group.SaleCount}");
            }
        }
    }
}

```

```

    }

    Console.Read();
}
}
}

```

This above example code will output the total amount of sales and the number of sales for each combination of country and year in our sales data. This approach can be very powerful for analyzing data across multiple dimensions, providing insights into sales trends, geographical performance, and temporal changes. When you run the above code, you will get the following output:

```

Country: USA, Year: 2023, Total Sales: 550, Sales Count: 3
Country: Canada, Year: 2023, Total Sales: 270, Sales Count: 2
Country: India, Year: 2023, Total Sales: 550, Sales Count: 1
Country: USA, Year: 2024, Total Sales: 400, Sales Count: 2
Country: Canada, Year: 2024, Total Sales: 100, Sales Count: 1

```

Handling Hierarchical Data Example using LINQ GroupBy Method with Multiple Keys

Handling hierarchical data with LINQ's GroupBy method using multiple keys allows you to categorize and summarize data at different levels of a hierarchy. This can be very useful for organizing data into nested groups based on multiple attributes. Let's consider an example where we have a collection of Employee objects, and we want to group these employees first by their Department and then by their Job Title.

First, we define a class to represent the employee records. Create a class file named **Employee.cs** and copy and paste the following code. We have also defined one method that will return a list of employees.

```

using System.Collections.Generic;
namespace GroupByDemo
{
    public class Employee
    {
        public string Name { get; set; }
        public string Department { get; set; }
        public string JobTitle { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            List<Employee> employees = new List<Employee>
            {
                new Employee { Name = "John Doe", Department = "IT", JobTitle
"Developer" },
                new Employee { Name = "Jane Smith", Department = "IT", JobTitl
"Developer" },
                new Employee { Name = "Mary Johnson", Department = "IT", JobTi
Tester" },
            }
        }
    }
}

```

```

        new Employee { Name = "James Brown", Department = "HR", JobTitle = "Recruiter" },
        new Employee { Name = "Sara Taylor", Department = "IT", JobTitle = "Tester" },
        new Employee { Name = "Steve Smith", Department = "HR", JobTitle = "Recruiter" },
        // Add more employees here
    };
    return employees;
}
}
}

```

To group these employees by Department and then by JobTitle, we can use the GroupBy method with a composite key. So, modify the Program class as follows:

```

using System;
using System.Linq;
namespace GroupByDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var groupedEmployees = Employee.GetAllEmployees()
                .GroupBy(e => new { e.Department, e.JobTitle })
                .Select(group => new
                {
                    Department = group.Key.Department,
                    JobTitle = group.Key.JobTitle,
                    Employees = group.ToList()
                });
            foreach (var group in groupedEmployees)
            {
                Console.WriteLine($"{group.Department} - {group.JobTitle}");
                foreach (var employee in group.Employees)
                {
                    Console.WriteLine($"  {employee.Name}");
                }
            }

            Console.Read();
        }
    }
}

```

In this example:

- GroupBy is used to create groups based on a composite key consisting of Department and JobTitle. This is achieved by creating an anonymous type with these two properties.

- The result of GroupBy is then projected using Select into a new anonymous type that includes the Department, JobTitle, and a list of employees belonging to that subgroup.
- Finally, the groups and their members are printed out, showing how the data is organized first by department and then by job title within each department.

When you run the above code, you will get the following output:

```
IT - Developer
    John Doe
    Jane Smith
IT - QA Tester
    Mary Johnson
    Sara Taylor
HR - Recruiter
    James Brown
    Steve Smith
```

In the next article, I will discuss the [LINQ ToLookup Method in C#](#) with Examples. In this article, I explain using the **LINQ GroupBy Method with Multiple Keys in C#** with Examples. I hope you enjoy this article and understand the need and use of the LINQ GroupBy with Multiple Keys in C#.